Loesungsskizze — Theorie (Aufgabe 3)

> *Lehrerexemplar. Punkteverteilung wie in der Angabe. Alternative gleichwertige Antworten sind moeglich; entscheidend ist, dass die zentralen Begriffe richtig verwendet werden.*

Block A — Programmiersprachen & Parser (8 P)

A1. ABNF-Grammatik der `WIEDERHOLE`-Anweisung (3 P)

```
wiederhole-stmt = "WIEDERHOLE" zahl "MAL" "{" *anweisung "}"
anweisung       = position-stmt / bewege-stmt / farbe-stmt / dicke-stmt
                / wiederhole-stmt / wenn-stmt
zahl            = ["-"] 1*DIGIT
DIGIT           = %x30-39
```

Bewertungsschluessel:

- 1 P: Regel `wiederhole-stmt` korrekt mit Schluesselwoertern, Klammern und Sternoperator
- 1 P: `<anweisung>` als Verweis (rekursiv), nicht ausformuliert
- 1 P: ABNF-Notation korrekt verwendet (`*` fuer 0..n, `[ ]` optional, Strings in `""`)

Auch akzeptabel: EBNF mit `{ ... }` fuer Wiederholung und `|` fuer Alternative — nicht beides mischen.

A2. Chomsky-Hierarchie (2 P)

- **Tokenisierung: Typ 3 (regulaer)** — jedes Token (Schluesselwort, Zahl, Klammer) laesst sich durch einen regulaeren Ausdruck bzw. einen endlichen Automaten erkennen. Es gibt keine geschachtelten Strukturen auf Token-Ebene. *(1 P)*
- **Statement-Grammatik: Typ 2 (kontextfrei)** — geschachtelte `{ ... }`-Bloecke (z. B. `WIEDERHOLE` in `WIEDERHOLE`) erfordern beliebig tiefe Verschachtelung, also Klammerstruktur. Das ist mit einem endlichen Automaten nicht erkennbar, sondern braucht einen Kellerautomaten. *(1 P)*

A3. Tokenisierung (1 P)

Eingabe: `WIEDERHOLE 4 MAL { BEWEGE -10 20 }`

#	TokenType	Value
1	Wiederhole	`WIEDERHOLE`
2	Number	`4`
3	Mal	`MAL`
4	Lkl	`{`
5	Bewege	`BEWEGE`
6	Number	`-10`
7	Number	`20`
8	Rkl	`}`

Volle Punkte, wenn alle 8 Tokens mit korrektem Typ vorhanden sind. Whitespace erzeugt keine Tokens.

A4. AST-Baum (2 P)

```
Programm
└── WiederholeStmt (n = 3)
    ├── BewegeStmt (10, 0)
    └── WennStmt (POSX > 50)
        └── DANN-Block
            └── FarbeStmt (Rot)
```

- 1 P fuer korrekte Wurzel + Schachtelung `WIEDERHOLE → { BEWEGE, WENN }`
- 1 P fuer korrekte WENN-Struktur mit DANN-Block (kein SONST hier, nicht zwingend)

Block B — Graphentheorie & Algorithmen (10 P)

> *Annahme: der Beispielgraph aus dem Angabezettel ist (zur Korrektur): Knoten A, B, C, D, E, F. Kanten: A–B(2), A–C(5), B–C(1), B–D(7), C–D(3), C–E(6), D–E(4), D–F(8), E–F(2). Bei abweichendem Beispielgraphen analog beurteilen.*

B1. Begriffe (3 P, 6 × ½ P)

- **Grad von C:** 4 (Nachbarn A, B, D, E).
- **Bipartit:** nein. Das Dreieck A–B–C hat ungerade Laenge 3 — bipartite Graphen enthalten keine Kreise ungerader Laenge.
- **Zentraler Knoten:** Knoten mit kleinster Exzentrizitaet (kleinster maximaler Abstand zu allen anderen). Im Beispiel: **C** (e(C) = max(0, 2, 1, 3, 6, 8) … berechnet 7).
- **Schnittknoten / Artikulation:** ein Knoten, dessen Entfernung den Graphen in zwei zusammenhanglose Teilgraphen zerlegt. Im Beispiel **kein** Schnittknoten vorhanden (Graph ist 2-fach zusammenhaengend).
- **Aufspannender Baum:** mindestens **n − 1 = 5** Kanten bei 6 Knoten.
- **Weg / Kreis / Zyklus:**
  - Weg = Folge von Kanten zwischen zwei Knoten, **kein** Knoten doppelt.
  - Kreis = Weg, der am Startknoten endet (geschlossener Weg).
  - Zyklus = synonym zu Kreis (insbesondere im gerichteten Fall).

B2. Dijkstra-Trockenlauf von A (4 P)

Iteration	finalisiert	dist A	dist B	dist C	dist D	dist E	dist F	Vorgaenger-Aktualisierung
Init	—	0	∞	∞	∞	∞	∞	—
1	A (0)	0	2	5	∞	∞	∞	B←A, C←A
2	B (2)	0	2	**3**	9	∞	∞	C←B (2+1=3), D←B (2+7=9)
3	C (3)	0	2	3	**6**	9	∞	D←C (3+3=6), E←C (3+6=9)
4	D (6)	0	2	3	6	9	14	F←D (6+8=14), E unveraend.
5	E (9)	0	2	3	6	9	**11**	F←E (9+2=11)
6	F (11)	0	2	3	6	9	11	fertig

Endgueltige Distanzen von A: 0, 2, 3, 6, 9, 11. *(je 1 P fuer korrekte Auswahl-Reihenfolge, korrekte Distanzen, korrekte Vorgaenger, saubere Tabellenform)*

B3. Permutationen — Komplexitaet (1 P)

- **N = 6** → 6! = **720** Permutationen
- **N = 8** → 8! = **40 320** Permutationen
- Die Aufgabe begrenzt auf N ≤ 6, weil die Anzahl der zu pruefenden Touren faktoriell waechst (`O(n!)`). Schon bei N = 8 sind es ueber 40 000 Touren, jeweils mit Dijkstra zwischen Stationspaaren — die Laufzeit waere fuer eine Live-Antwort zu hoch.

B4. Pseudocode rekursiver Permutationsgenerator (2 P)

```
function permutiereAlle(start, rest):
    ergebnis ← leere Liste
    aktuell  ← [start]
    permutiere(rest, aktuell, ergebnis)
    return ergebnis

function permutiere(rest, aktuell, ergebnis):
    wenn rest ist leer:
        ergebnis.add(kopie von aktuell)
        return
    fuer i = 0 bis rest.Count - 1:
        x ← rest[i]
        rest.entfernen(i)
        aktuell.anhaengen(x)
        permutiere(rest, aktuell, ergebnis)
        aktuell.entferneLetztes()
        rest.einfuegen(i, x)
```

- 1 P fuer korrekte Rekursion mit Backtracking-Schritt (entfernen + einfuegen)
- 1 P fuer fixiertes erstes Element (`start` wird nicht permutiert)

Block C — UML & Software-Pattern (8 P)

C1. Klassendiagramm (4 P)

```
                ┌──────────────────────┐
                │     <<abstract>>     │
                │      Expression      │
                ├──────────────────────┤
                │ + Errors : List<string>│  (static)
                │ + Parse(t : List<Token>) : void  (abstract)
                │ + Run(z : Zeichner)    : void  (abstract)
                └──────────▲───────────┘
                           │
   ┌──────────┬──────────┬──────────┬──────────┬────────────┐
   │          │          │          │          │            │
PositionStmt BewegeStmt FarbeStmt DickeStmt WiederholeStmt   WennStmt
                                            ◇ kinder        ◇ dann
                                                           ◇ sonst
                                            0..*            0..*
                                            Expression      Expression
```

Mindestbestandteile fuer volle Punktzahl:

- **abstrakte Basisklasse** `Expression` mit `Parse` / `Run` *(1 P)*
- **mind. 5 konkrete Subklassen** *(1 P)*
- **Composite-Beziehung**: `WiederholeStmt` und `WennStmt` haben eine `0..*` -Aggregation/Komposition zu `Expression` *(1 P)*
- **Sichtbarkeiten** (`+` public, `-` private, `#` protected) korrekt verwendet *(1 P)*

C2. Software-Pattern (3 P, je 1 P)

1. **Composite** — `WiederholeStmt` und `WennStmt` enthalten `List<Expression>` und behandeln einzelne Statements und ganze Bloecke einheitlich (gleiche Basis-Klasse `Expression`).

2. **Interpreter** — die gesamte Klassenhierarchie ist ein klassischer Interpreter: `Expression.Run(Zeichner)` interpretiert den AST direkt zur Laufzeit. Jede Sprachregel ist eine Klasse mit eigener `Run`-Implementierung.
3. **Observer** — `Transfer` informiert ein registriertes `Receiver`-Objekt ueber eintreffende Nachrichten via `ReceiveMessage(...)`. Server und Client sind Observer ihrer jeweiligen `Transfer`-Instanz.

Auch akzeptabel: **Singleton** (eine `GraphData`-Instanz im Server), **Iterator** (`foreach` ueber Stationen / Verbindungen), **Factory** (zentrale Stelle, die Statement-Klassen aus Tokens erzeugt). Wenn Schueler ein passendes Pattern korrekt zuordnen, voller Punkt.

C3. UML-Beziehungstypen (1 P)

Typ	Symbol	Bedeutung
Aggregation	leere Raute am Ganzen, durchgezogen	„hat-ein" mit unabhaengiger Lebensdauer (Teil ueberlebt das Ganze)
Komposition	gefuellte Raute am Ganzen, durchgezogen	starke „hat-ein"-Beziehung; das Teil stirbt mit dem Ganzen
Generalisation	leere Pfeilspitze am Oberbegriff	„ist-ein" / Vererbung
Abhaengigkeit	gestrichelter Pfeil	„verwendet kurzzeitig" (z. B. Parameter, lokale Variable)

½ P pro korrekte Bedeutung; ¼ P pro korrekt benanntem Symbol. Volle Punkte wenn alle vier sauber unterschieden werden.

Block D — Netzwerk, ORM, Threads, XML (4 P)

D1. XML-Serialisierung (2 P)

Drei Regeln (je ⅓ P, davon volle 1 P):

1. Klasse muss **public** sein.
2. Klasse braucht einen **parameterlosen Standard-Konstruktor**, sonst kann der Deserializer das Objekt nicht erzeugen.
3. Felder / Properties muessen **public** und schreibbar sein. Nur dann werden sie von `XmlSerializer` beruecksichtigt; mit `[XmlIgnore]` lassen sich einzelne Felder ausnehmen.

Polymorphie-Problem (1 P): Der `XmlSerializer` kennt zur Konstruktionszeit nur den **deklarierten** Typ. Wenn ein Feld `Tier` ist, aber zur Laufzeit ein `Hund`-Objekt haengt, weiss der Serializer nichts vom Subtyp und wirft eine Exception. **Loesung:** mit dem Attribut `[XmlInclude(typeof(Hund))]` an der Basisklasse alle erlaubten Subtypen anmelden, oder per `[XmlElement(typeof(Hund))]` / Wrapper-Attributen am Feld.

D2. Threads (1 P)

- **Client (1 Verbindung):** 2 Threads — UI-Thread (WPF-Dispatcher) + Empfangsthread (`Transfer.Run` blockiert auf `ReadLine`).
- **Server (2 Clients):** 1 Accept-Thread + 2 Empfangsthreads = **3 anwendungsrelevante Threads**.

Warum pro TCP-Verbindung ein eigener Thread? `ReadLine` blockiert, bis Daten ankommen. Mit nur einem Thread wuerde der Server beim Warten auf Verbindung A keine Anfragen von Verbindung B mehr lesen. Ein eigener Thread pro Verbindung erlaubt parallele I/O ohne nicht-blockierende Sockets.

D3. ORM und Datenbinding (1 P)

- **ORM** = Object-Relational Mapping: ein Framework bildet Datenbanktabellen auf Klassen (`[Table]` / `[Column]`-Annotationen) und Datensaetze auf Objekte ab. In Aufgabe 2 verwendet der Client **linq2db**, um `RouteEintrag`-Objekte direkt in die SQLite-Tabelle `RouteEintraege` zu speichern und wieder zu lesen — ohne manuelles SQL.
- **Datenbinding** verbindet UI-Elemente direkt mit Properties eines Objektes; AEnderungen an einer Seite werden automatisch auf die andere uebernommen. Damit die UI **nach** dem Anlegen des Bindings auf Aenderungen am Objekt reagiert, muss die Klasse die Schnittstelle `INotifyPropertyChanged` implementieren und im Setter `PropertyChanged` ausloesen.

Punktezusammenfassung

Block	Maximalpunkte
A — Sprache & Parser	8
B — Graphentheorie & Algorithmen	10
C — UML & Software-Pattern	8
D — Netzwerk, ORM, Threads, XML	4
Gesamt Theorie	**30**